



PeruvianMarket

PROYECTO DE CRIPTOGRAFÍA APLICADA

Informe Técnico: Criptografía de extremo a extremo en un mercado de predicción

Plataforma web de mercados de predicción, apuestas P2P privadas y casino entre amigos, construida sobre cinco primitivos criptográficos reales: ECDSA, ECDH, Ed25519, AES-256-GCM y PBKDF2.

Curso: Criptografía · UTEC

Aplicación: Next.js 15 + Supabase · desplegada en Raspberry Pi vía Tailscale Funnel

Demo: <https://bot.tail48662d.ts.net/login>

Naturaleza: laboratorio funcional — no maneja dinero real

— Índice

1	Motivación	3
2	Descripción del sistema y arquitectura	3
3	Componentes criptográficos y flujos	4
4	Justificación de algoritmos y librerías	7
5	Seguridad práctica y análisis de amenazas	7
6	Limitaciones conocidas	8
7	Evaluación de resultados	8
8	Stack, despliegue y estructura	9

1. Motivación

Frente a plataformas de mercados de predicción como **Polymarket**, falta una **alternativa peruana** que aplique los mismos conceptos criptográficos. PeruvianMarket nace de esa idea: una plataforma de apuestas basada en su propia moneda blockchain (CHC), que combina mercados de predicción, apuestas P2P privadas y juegos de casino.

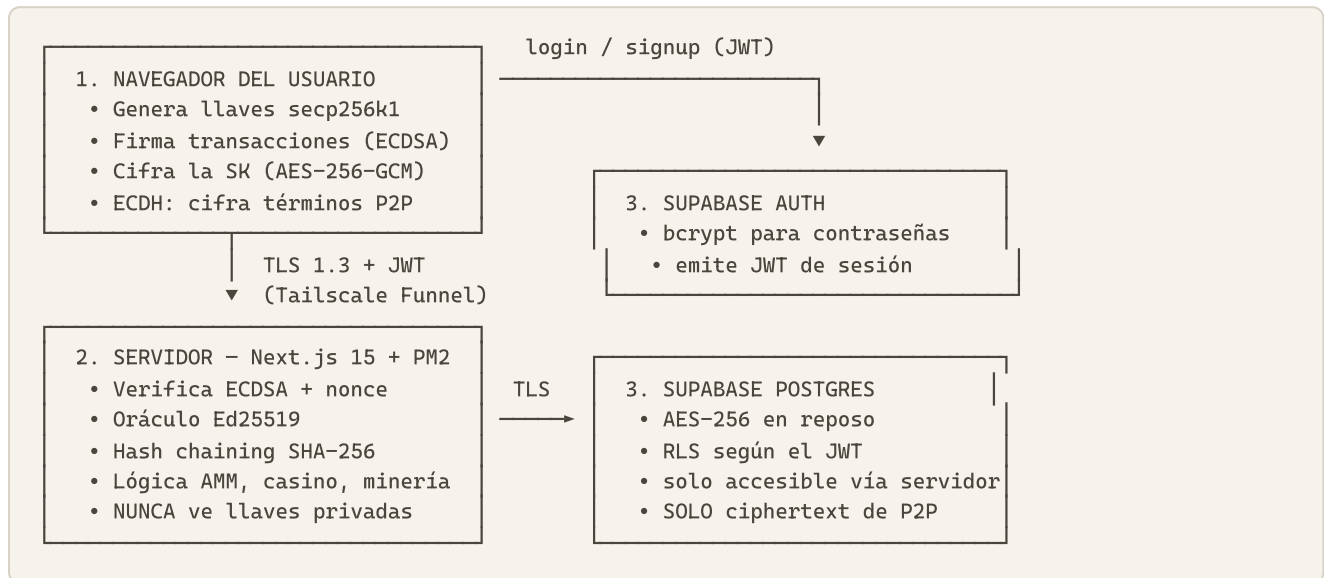
En el fondo, un grupo de amigos que quiere apostar entre sí enfrenta problemas de confianza que la criptografía resuelve mejor que cualquier acuerdo verbal. En vez de depender de que alguien "haga de banco" o "haga de árbitro", cada garantía se sustituye por un mecanismo criptográfico verificable.

PROBLEMA	PREGUNTA	SOLUCIÓN CRIPTOGRÁFICA
Custodia	¿Quién guarda el dinero y con qué autoridad lo mueve?	Cada usuario posee su par de llaves; toda transferencia exige su firma ECDSA .
Privacidad	¿Cómo aposto algo personal sin que un tercero lo lea?	Términos cifrados con ECDH + AES-256-GCM entre las dos partes.
Arbitraje	¿Quién decide quién ganó y cómo lo pruebo?	Veredictos firmados; en disputa, un oráculo Ed25519 resuelve de forma auditable.
Historial	¿Cómo demuestro que algo ocurrió sin alteración?	Ledger con hash chaining SHA-256 verificable en un clic.

El objetivo académico es aplicar los primitivos vistos en clase —simétricos, asimétricos, hashes, derivación de claves, firmas y acuerdo de claves— en un caso de uso realista, tomando decisiones concretas de diseño y seguridad.

2. Descripción del sistema y arquitectura

PeruvianMarket es una aplicación web donde los usuarios gestionan una wallet propia de CHCoin (CHC) y pueden: crear **mercados de predicción** (AMM $x \cdot y = k$ y parimutuel), abrir **mercados P2P privados** cifrados extremo a extremo, jugar en un **casino** de 7 juegos con *house edge* configurable, librar **batallas Pokémon** con apuestas (motor oficial de Pokémon Showdown), minar CHC y auditar todo en un explorador de blockchain.



Principio de diseño central: la llave privada de un usuario nunca abandona su navegador —ni en claro ni cifrada—. El servidor es un verificador de firmas y un ejecutor de lógica, jamás un custodio de secretos.

3. Componentes criptográficos y flujos

Se implementan cinco primitivas cubriendo criptografía simétrica, asimétrica, funciones hash, derivación de claves y acuerdo de claves. Ninguna fue programada desde cero.

3.1 · Wallets estilo Bitcoin — secp256k1 (asimétrica)

La dirección se deriva igual que en Bitcoin (P2PKH), con byte de versión propio `0x35` (prefijo "N"):

```
privKey (32 B, CSPRNG)
├─> pubKey comprimida (33 B)
├─> hash160 = RIPEMD-160(SHA-256(pubKey))
├─> versioned = 0x35 || hash160
├─> checksum = SHA-256(SHA-256(versioned))[:4]
└─> dirección = Base58(versioned || checksum) → "N ... "
```

El checksum de 4 bytes detecta direcciones mal tipeadas antes de mover fondos. Archivo: `web/src/lib/crypto.ts`.

3.2 · Firma de transacciones — ECDSA

Toda transferencia se firma en el cliente sobre un **JSON canónico** (claves ordenadas alfabéticamente, de modo que cliente y servidor computan el mismo hash) y se verifica server-side con triple control:



El mensaje firmado incluye `{type, from, to, amount, nonce, timestamp}`; alterar cualquier campo invalida la firma.

3.3 · Bóveda de llave privada — AES-256-GCM + PBKDF2 (simétrica + KDF)

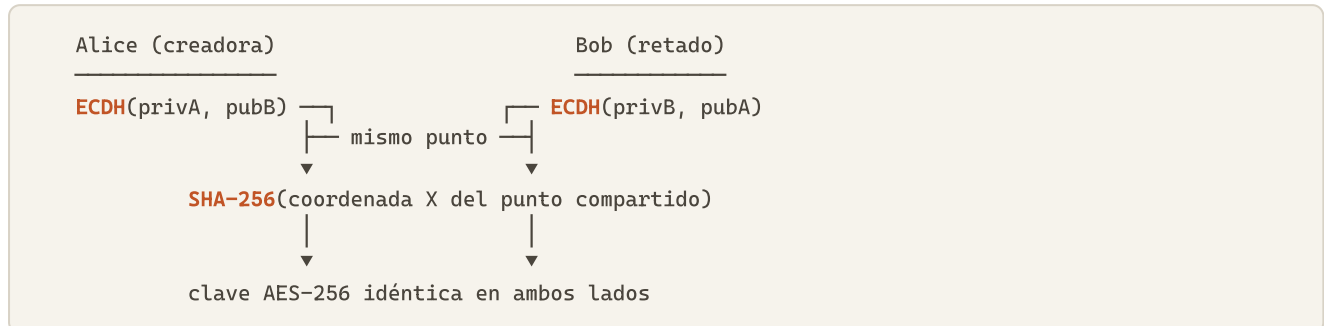
La llave privada se cifra en el navegador antes de tocar disco (`localStorage`):

PARÁMETRO	VALOR	JUSTIFICACIÓN
Cifrado	AES-256-GCM	Cifrado autenticado: confidencialidad + integridad (tag 128 bits) en una operación.
KDF	PBKDF2-SHA256, 600 000 iter.	Recomendación OWASP 2023; ralentiza fuerza bruta offline.
Salt	16 B CSPRNG, único	Evita rainbow tables.
IV / nonce	12 B CSPRNG, único	Nunca se reutiliza con la misma clave (requisito de GCM).
Formato	v2: Base64(salt iv ct+tag)	Versionado: migración transparente desde blobs de 100k iteraciones.

El servidor nunca ve este blob; sin la contraseña maestra es inútil, y GCM detecta cualquier manipulación.

3.4 · Mercados P2P privados — ECDH + AES-GCM + doble firma (cifrado híbrido)

El aporte más completo del proyecto. Ambas partes derivan la **misma clave simétrica sin transmitirla jamás**, gracias a la propiedad del acuerdo de claves Diffie-Hellman sobre curva elíptica:



Sobre esa clave se ejecuta el protocolo de cuatro fases:

1. CREAR Alice cifra los términos (AES-256-GCM, clave ECDH)
terms_hash = SHA-256(ciphertext)
firma ECDSA sobre {P2P_CREATE, from, to, amount, terms_hash, deadline, nonce, ts}
→ deposita en escrow. El servidor guarda SOLO ciphertext.
2. ACEPTAR Bob descifra localmente (ECDH es simétrico), revisa, firma ECDSA sobre el MISMO terms_hash → acuerdo probado
→ deposita. Pot en escrow (2× amount).
3. VEREDICTO Cada parte firma su veredicto.
coinciden → pago automático al ganador
difieren → estado "disputa"
4. ORÁCULO Si hay disputa o vence el plazo, el oráculo resuelve y FIRMA con Ed25519. La firma queda auditable.

El anclaje de ambas firmas a `SHA-256(ciphertext)` garantiza **no repudio**: ninguna parte puede negar que aceptó exactamente esos términos. Archivos: `crypto.ts`, `p2p.ts`, `app/p2p/`.

3.5 · Oráculo de resoluciones — Ed25519 (asimétrica)

El oráculo firma cada resolución con un mensaje canónico:

```
mensaje = "p2p:{marketId}:{winnerAddress}:{timestamp}"  
firma   = Ed25519_sign(mensaje, ORACLE_PRIVATE_KEY) // solo en el servidor
```

¿Por qué Ed25519 y no ECDSA de nuevo?

- **Firmas deterministas:** sin nonce aleatorio por firma, eliminando una clase entera de fallas que rompió wallets con ECDSA mal implementado.
- **Verificación pública:** la llave pública del oráculo se expone; cualquiera valida que una resolución es auténtica.
- **Separación de dominios:** los usuarios firman con secp256k1, el árbitro con Ed25519 — una llave comprometida no cruza roles.

3.6 · Blockchain con hash chaining — SHA-256

Las transacciones se sellan en bloques de 5, encadenados por hash:

```
block_hash = SHA-256( número | prev_hash | tx1:tipo:from:to:monto ; ... ; tx5 )
```

Alterar cualquier transacción sellada o bloque rompe la cadena desde ese punto. El explorador incluye un botón **"Verificar cadena"** que recompute todos los hashes y reporta el primer bloque inválido si la base de datos fue manipulada. Archivo: [web/src/lib/blockchain.ts](#).

4. Justificación de algoritmos y librerías

Se usan exclusivamente librerías probadas y auditadas; **no se implementó ningún primitivo criptográfico desde cero** (requisito del curso). Todos los algoritmos siguen estándares actuales: no se usan hashes rotos (MD5, SHA-1) ni modos débiles (DES, ECB).

LIBRERÍA	USO	POR QUÉ
@noble/secp256k1	ECDSA (firmas) + ECDH (acuerdo de claves)	Auditada de forma independiente; usada en producción por MetaMask y otros.
@noble/ed25519	Firmas del oráculo	Misma familia minimalista, sin dependencias.
@noble/hashes	SHA-256, SHA-512, RIPEMD-160, HMAC	Implementaciones de tiempo constante, revisadas.
WebCrypto crypto.subtle	AES-256-GCM + PBKDF2	Primitivos nativos del navegador, no reimplementados en JS.
@pkmn/sim	Motor de batallas Pokémon	Código oficial de Pokémon Showdown (licencia MIT).

5. Seguridad práctica y análisis de amenazas

Protección de datos **en reposo** (AES-256 en Postgres, bóveda AES-GCM en el cliente, contraseñas bcrypt) y **en tránsito** (TLS 1.3 extremo a extremo, autenticación por JWT).

Herramientas de análisis estático de seguridad

Se aplicaron dos escáneres complementarios sobre el código:

HERRAMIENTA	QUÉ ANALIZA	RESULTADO
Semgrep (SAST)	Patrones inseguros en el código fuente — 225 reglas de la comunidad + reglas TS	0 hallazgos · 22 reglas ejecutadas · 88 archivos escaneados
npm audit	Vulnerabilidades conocidas (CVE/GHSA) en dependencias	0 vulnerabilidades
tsc --noEmit	Análisis estático de tipos	0 errores

Semgrep (Static Application Security Testing) escaneó los 88 archivos rastreados por git contra su registro de reglas de seguridad y no encontró ningún patrón vulnerable (o *findings*, o *blocking*). **npm audit** detectó y se corrigió previamente una vulnerabilidad moderada en **postcss** mediante *overrides*, sin degradar Next.js.

AMENAZA	DEFENSA	ESTADO
Intercepción de tráfico	TLS 1.3; la llave privada nunca viaja	MITIGADO
Replay de transacción firmada	Nonce monotónico dentro del mensaje firmado	MITIGADO
Resolución falsificada	Firma Ed25519 del oráculo, verificable	MITIGADO
Fuga completa de la base de datos	Sin llaves privadas; bcrypt; firmas no reutilizables	MITIGADO
Servidor espía apuestas P2P	Cifrado E2E ECDH + AES-GCM; solo ciphertext	MITIGADO
Una parte niega el acuerdo P2P	Firmas ECDSA ancladas a SHA-256(ciphertext)	MITIGADO
Admin edita el historial	Hash chaining SHA-256 + verificación pública	MITIGADO
Doble gasto / doble cobro	Débito atómico + claims condicionales por estado	MITIGADO
XSS exfiltra el blob cifrado	AES-GCM + PBKDF2 $\times 600k \rightarrow$ solo fuerza bruta offline	DEPENDE DE LA CLAVE

6. Limitaciones conocidas y mejoras futuras

Limitaciones actuales:

- **Casino custodial:** las apuestas de casino se autorizan por sesión JWT, no por firma ECDSA (decisión de UX para partidas rápidas). Las transferencias y los mercados P2P sí exigen firma.
- **Oráculo centralizado:** el árbitro de disputas es una única llave Ed25519 del administrador.
- **Confianza en el servidor:** aunque las llaves privadas nunca lo tocan, el servidor ejecuta la lógica de casino y mercados, por lo que aún requiere cierta confianza.

Mejoras planificadas (líneas de trabajo futuro):

MEJORA	DETALLE
Mejorar el mecanismo de oráculo de resolución	Pasar de una llave única a un esquema de umbral (t-de-n) o múltiples oráculos, reduciendo el punto único de confianza en el arbitraje.
Migración de KDF a Argon2id	PBKDF2 $\times 600k$ cumple OWASP 2023, pero Argon2id es resistente a GPU/ASIC y sería superior para derivar la clave de la bóveda.
Reducir la confianza en el servidor	Mover más lógica a verificación criptográfica del lado del cliente / on-chain, minimizando lo que el servidor puede alterar.
Mejora contra ataques DoS	Rate limiting en los endpoints de firma y de juego para frenar abuso y fuerza bruta en línea.

7. Evaluación de resultados

Logros verificables:

- **Cinco primitivas** funcionando en un producto real desplegado: ECDSA, ECDH, Ed25519, AES-256-GCM y PBKDF2, más SHA-256/512 y RIPEMD-160.
- **Cifrado híbrido completo** (KEM/DEM conceptual): ECDH acuerda la clave, AES-GCM cifra los datos — los mercados P2P privados.
- **No repudio y privacidad E2E** demostrables en vivo: se abre la base de datos y se muestra que los términos son ilegibles, luego se descifran en la interfaz con la contraseña del usuario.
- **Concurrencia segura:** operaciones de dinero atómicas, verificadas para múltiples usuarios simultáneos.
- **Auditabilidad:** blockchain con verificación de integridad de un clic.

Fallas y aprendizajes. La primera versión de la "blockchain" usaba hashes falsos derivados del ID de transacción (agrupación visual, sin garantía real); se corrigió a hash chaining verdadero. El límite de memoria de PM2 causaba reinicios que cancelaban batallas; se diagnosticó y ajustó. Ambos casos ilustran la diferencia entre *"parece criptográfico"* y *"es criptográficamente sólido"*.

8. Stack, despliegue y estructura

Frontend Next.js 15 (App Router) + TypeScript + Tailwind CSS

- Backend** API routes de Next.js sobre Node.js
- Base** Supabase (Postgres + Auth + Row Level Security)
- datos**
- Hosting** Raspberry Pi + PM2, expuesta con Tailscale Funnel (HTTPS sin abrir puertos)

Estructura del repositorio (resumen):

```
proyecto11/
├── web/
│   ├── src/lib/crypto.ts      Aplicación Next.js
│   ├── src/lib/oracle.ts     secp256k1, ECDH, AES-GCM, PBKDF2, direcciones
│   ├── src/lib/p2p.ts        firmas Ed25519 del oráculo
│   ├── src/lib/blockchain.ts lógica de mercados P2P
│   ├── src/app/api/          hash chaining SHA-256
│   └── sql/                  rutas de backend (wallet, p2p, casino, pokemon)
│                               migraciones de base de datos
├── deploy-rpi/              scripts y guías de despliegue
├── PROYECTO.md              resumen ejecutivo
└── README.md                informe en Markdown
```

Nota de seguridad: `web/.env.local` (llaves de Supabase y del oráculo) y cualquier `*_key.json` no están versionados en el repositorio — deben copiarse manualmente o recrearse desde las plantillas.

Proyecto académico — no maneja dinero real. Los nombres y sprites de Pokémon son propiedad de Nintendo / The Pokémon Company; su uso aquí es sin fines de lucro, en el mismo espíritu que Pokémon Showdown.